ГУАП

КАФЕДРА № 41

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

Ассистент	_____	А. С. Раскопина
должность	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 6

Программная реализация хранимых процедур

по курсу: Базы данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417	_____	М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Лабораторная работа № 6

Программная реализация хранимых процедур

Цель работы

Разработать хранимые процедуры для базы данных «ГИБДД» на сервере MySQL; применить входные параметры, локальные и пользовательские переменные, ветвления и цикл; проверить корректные вызовы и обработку ошибочных параметров.

1 Вариант и исходная база данных

Вариант 13 — «ГИБДД». Государственная инспекция безопасности дорожного движения Хоббитонии регистрирует автомобили граждан и учитывает ДТП. Для автомобиля хранятся марка, мощность, максимальная скорость, год выпуска, стоимость и текущий владелец. ДТП характеризуется датой, временем, адресом и составом участвующих автомобилей. Один гражданин может владеть несколькими автомобилями и продавать их другим гражданам.

Используется схема gibdd из работ № 4 и № 5: citizens — граждане; cars — автомобили; accidents — ДТП; accident_cars — участия автомобилей; sales — продажи. Контрольный набор: 10 граждан, 10 автомобилей, 5 ДТП, 10 участий и 5 продаж. Проверка выполнена на MySQL 8.0.46 с ONLY_FULL_GROUP_BY и STRICT_TRANS_TABLES. Структура и значения соответствуют данным пятой работы.

На рисунке 1 приведена схема из предоставленного отчёта № 5. Все шесть связей перечислены в таблице 1; полные имена полей, частично сокращённые исходным редактором на изображении, указаны в таблице 2.

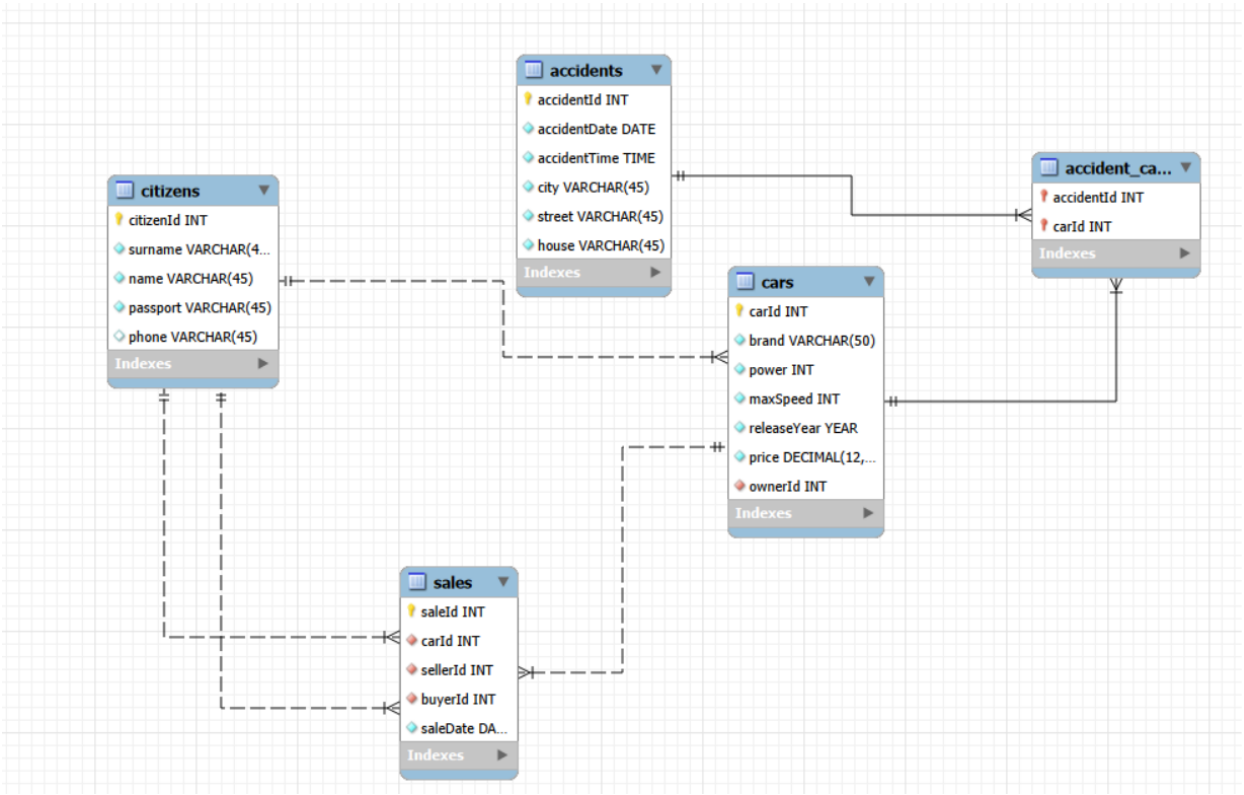


Рисунок 1 — Схема gibdd в MySQL Workbench из отчёта № 5

Таблица 1 — Связи базы данных

Внешний ключ	Ссылка
cars.ownerId	citizens.citizenId
accident_cars.accidentId	accidents.accidentId
accident_cars.carId	cars.carId
sales.carId	cars.carId
sales.sellerId	citizens.citizenId
sales.buyerId	citizens.citizenId

Таблица 2 — Поля исходных таблиц

Таблица	Поля
citizens	citizenId INT PK; surname VARCHAR(45); name VARCHAR(45); passport VARCHAR(45) UNIQUE; phone VARCHAR(45)
cars	carId INT PK; brand VARCHAR(50); power INT; maxSpeed INT; releaseYear YEAR; price DECIMAL(12,2); ownerId INT FK
accidents	accidentId INT PK; accidentDate DATE; accidentTime TIME; city VARCHAR(45); street VARCHAR(45); house VARCHAR(45)
accident_cars	accidentId INT PK/FK; carId INT PK/FK; составной PK (accidentId, carId)
sales	saleId INT PK; carId INT FK; sellerId INT FK; buyerId INT FK; saleDate DATE

cars.ownerId хранит текущего владельца. Процедура карточки не определяет, кто владел автомобилем на дату ДТП. Число ДТП показывает зарегистрированные участия и не характеризует виновность водителя.

2 Переменные и установка процедур

Локальные переменные объявлены через DECLARE и действуют внутри вызова. Пользовательские @lr6_status, @lr6_last_accident_count и @lr6_last_year_total сохраняют результат в текущем соединении после CALL. В учебной терминологии это переменные вне локального блока («глобальные» для подпрограммы), но технически они сессионные: другие соединения их не видят. Серверные переменные @@GLOBAL и @@SESSION — отдельный механизм; настройки всего сервера работа не изменяет. @@SESSION.sql_mode читается в контрольном запросе.

Для предотвращения использования устаревшего результата процедура в начале сбрасывает свою @-переменную в NULL и устанавливает ERROR. После успешного выполнения записываются итог и OK. @lr6_status относится к последней вызванной процедуре. Чужой итог не сбрасывается: после карточки следует читать @lr6_last_accident_count, после сводки — @lr6_last_year_total.

Файл 01_procedures.sql выполняется целиком в Workbench. DROP PROCEDURE IF EXISTS удаляет только одноимённые учебные процедуры; исходные таблицы не пересоздаются. DELIMITER — команда клиента для передачи тела с внутренними точками с запятой; после создания разделитель возвращается к «;». Для установки нужны права CREATE ROUTINE и, при замене, ALTER ROUTINE; для вызова — EXECUTE, а при стандартном SQL SECURITY DEFINER определитель должен иметь доступ к таблицам и CREATE TEMPORARY TABLES для сводки.

3 Процедура карточки автомобиля

Прикладное назначение lr6_car_card — быстро получить карточку автомобиля и число его участий в ДТП. Её может использовать инспектор при проверке автомобиля по внутреннему идентификатору. Единственный параметр IN p_car_id INT должен быть положительным и существовать в cars. Возвращается одна строка с автомобилем, текущим владельцем,

accident_count и accident_status; итог также записывается в @lr6_last_accident_count.

Алгоритм показан на рисунке 2. Сначала проверяются формат ID и существование записи. При нарушении выполняется SIGNAL SQLSTATE 45000 с диагностикой. Затем COUNT(*) подсчитывает участия автомобиля. Благодаря составному первичному ключу accident_cars одна машина не учитывается дважды в одном ДТП. CASE определяет отметку HAS_ACCIDENTS либо NO_ACCIDENTS. Листинг 1 реализует этот алгоритм и содержит пять поясняющих комментариев.

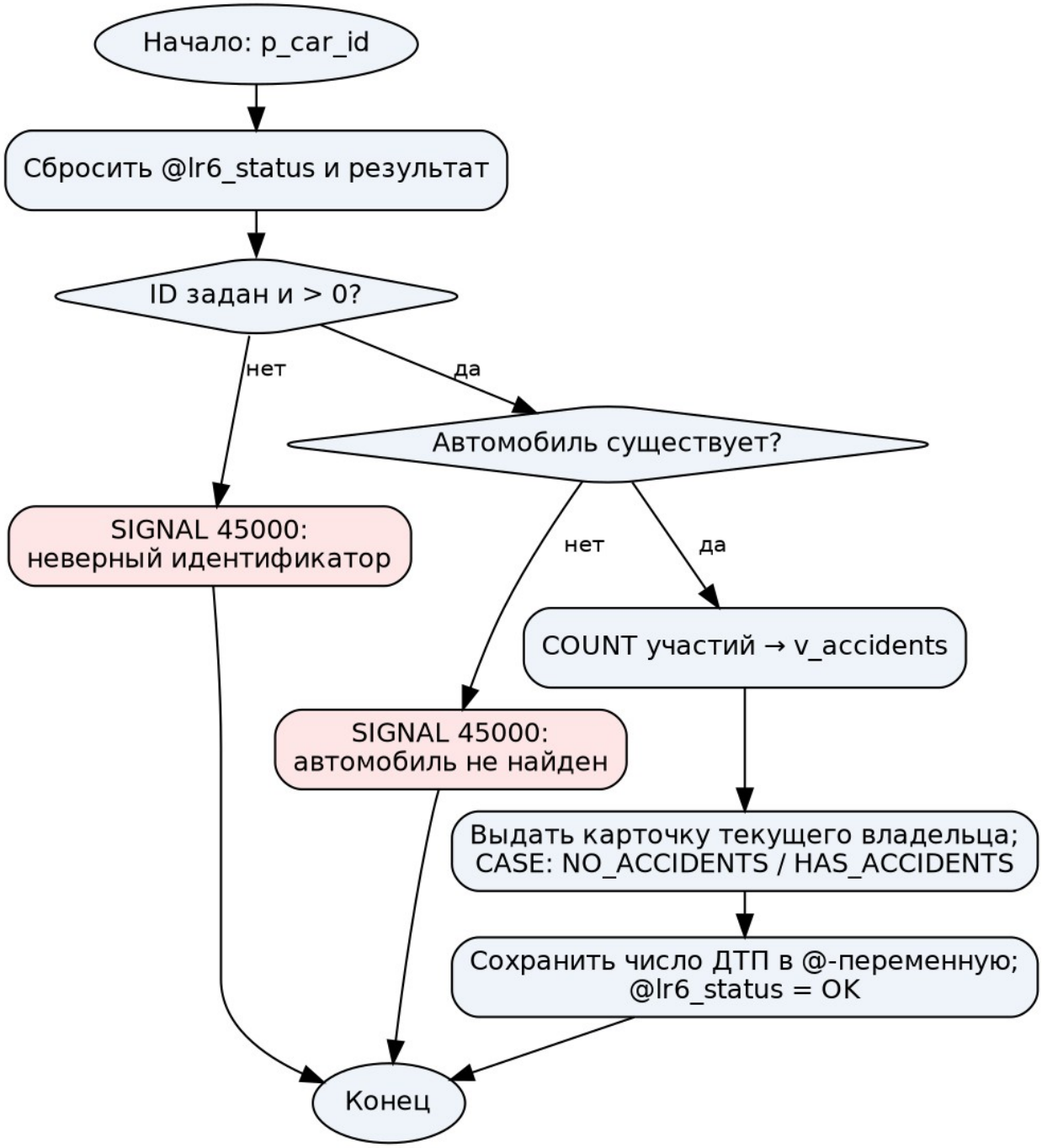


Рисунок 2 — Алгоритм lr6_car_card

Листинг 1 — Создание процедуры карточки

```
USE gibdd;
DELIMITER $$
CREATE PROCEDURE lr6_car_card(IN p_car_id INT)
READS SQL DATA
BEGIN
    DECLARE v_exists INT DEFAULT 0;
    DECLARE v_accidents INT DEFAULT 0;
    -- Сессионные результаты сбрасываются до проверки параметра.
    SET @lr6_status = 'ERROR', @lr6_last_accident_count = NULL;
    -- Ошибка формата идентификатора.
    IF p_car_id IS NULL OR p_car_id <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'carId must be a positive integer';
    END IF;
    SELECT COUNT(*) INTO v_exists FROM gibdd.cars WHERE carId = p_car_id;
    -- Отдельная ветка для отсутствующего автомобиля.
    IF v_exists = 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Car with this carId does not exist';
    END IF;
    SELECT COUNT(*) INTO v_accidents
    FROM gibdd.accident_cars WHERE carId = p_car_id;
    -- LEFT JOIN сохраняет карточку автомобиля даже без участий в ДТП.
    SELECT c.carId, c.brand, c.releaseYear, c.price,
           p.citizenId, p.surname, p.name,
           v_accidents AS accident_count,
           CASE WHEN v_accidents = 0 THEN 'NO_ACCIDENTS'
                ELSE 'HAS_ACCIDENTS' END AS accident_status
    FROM gibdd.cars AS c
    LEFT JOIN gibdd.citizens AS p ON p.citizenId = c.ownerId
    WHERE c.carId = p_car_id;
    -- Результат доступен после CALL в том же соединении.
    SET @lr6_last_accident_count = v_accidents, @lr6_status = 'OK';
END$$
DELIMITER ;
```

4 Процедура помесячного анализа ДТП

lr6_accidents_by_month нужна аналитику ГИБДД для сравнения активности по месяцам выбранного года. IN p_year INT принимает год от 1901 до 2155 включительно — это явное учебное ограничение, а не предел типа DATE. NULL и значения вне диапазона приводят к ошибке 45000. Результат содержит 12 строк, включая месяцы без ДТП; сумма передаётся через @lr6_last_year_total.

На рисунке 3 показан алгоритм. WHILE перебирает месяцы от 1 до 12. Для каждого месяца вычисляется дата начала и считается количество ДТП в полуоткрытом интервале [начало месяца; начало следующего). IF выбирает отметку NO_DATA либо HAS_DATA, итог накапливается в v_total. Оба присваивания в конце итерации обязательны: накопление формирует годовой итог, увеличение v_month завершает цикл.

Временная таблица lr6_month_result существует только в текущем соединении и удаляется после выдачи результата. Исходные таблицы не изменяются. Характеристика MODIFIES SQL DATA отражает INSERT во временную таблицу. EXIT HANDLER удаляет временный результат при SQL-ошибке, сбрасывает итог и повторно передаёт ошибку через RESIGNAL. Имя временной таблицы зарезервировано для работы № 6. Код показан в листинге 2; в нём пять поясняющих комментариев.

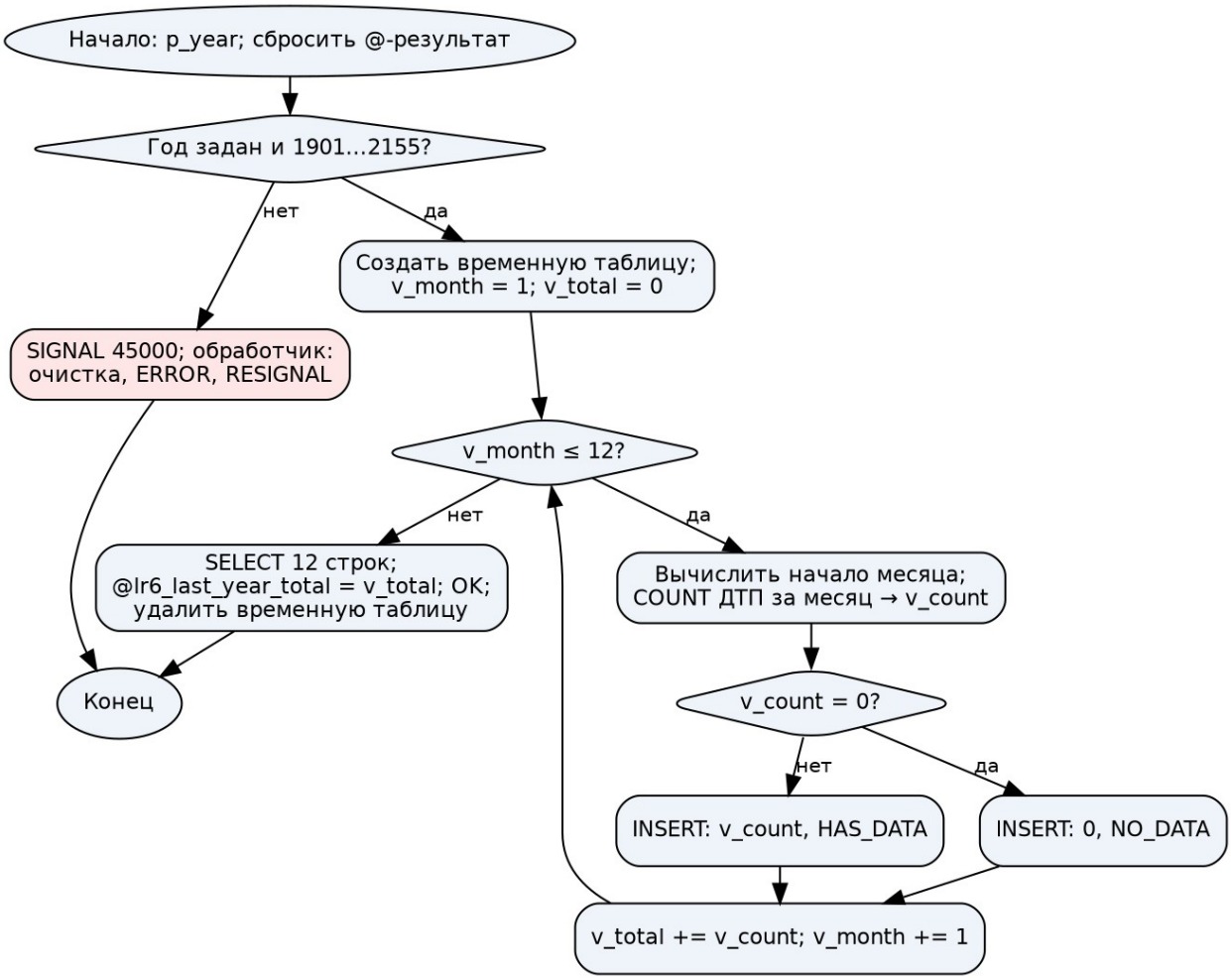


Рисунок 3 — Алгоритм lr6_accidents_by_month

Листинг 2 — Создание процедуры годовой сводки

```
USE gibdd;
DELIMITER $$
CREATE PROCEDURE lr6_accidents_by_month(IN p_year INT)
MODIFIES SQL DATA
BEGIN
    DECLARE v_month INT DEFAULT 1;
    DECLARE v_date DATE;
    DECLARE v_count INT DEFAULT 0;
    DECLARE v_total INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        -- Очистка временного результата при любой SQL-ошибке.
        DROP TEMPORARY TABLE IF EXISTS lr6_month_result;
        SET @lr6_status = 'ERROR', @lr6_last_year_total = NULL;
        RESIGNAL;
    END;
    SET @lr6_status = 'ERROR', @lr6_last_year_total = NULL;
    -- Ограничение диапазона соответствует принятому учебному диапазону лет.
    IF p_year IS NULL OR p_year < 1901 OR p_year > 2155 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Year must be between 1901 and 2155';
    END IF;
    DROP TEMPORARY TABLE IF EXISTS lr6_month_result;
    CREATE TEMPORARY TABLE lr6_month_result (
        month_no INT PRIMARY KEY,
        accident_count INT NOT NULL,
        month_status VARCHAR(10) NOT NULL
    );
    -- Цикл формирует все 12 месяцев, включая месяцы без ДТП.
    WHILE v_month <= 12 DO
```

```
        SET v_date = STR_TO_DATE(CONCAT(p_year, '-', LPAD(v_month, 2, '0'), '-01'),
'%Y-%m-%d');
        SELECT COUNT(*) INTO v_count FROM gibdd.accidents
        WHERE accidentDate >= v_date
          AND accidentDate < DATE_ADD(v_date, INTERVAL 1 MONTH);
        -- Обе ветки сохраняют строку с понятной отметкой активности.
        IF v_count = 0 THEN
            INSERT INTO lr6_month_result VALUES (v_month, 0, 'NO_DATA');
        ELSE
            INSERT INTO lr6_month_result VALUES (v_month, v_count, 'HAS_DATA');
        END IF;
        SET v_total = v_total + v_count;
        SET v_month = v_month + 1;
    END WHILE;
    SELECT p_year AS report_year, month_no, accident_count, month_status
    FROM lr6_month_result ORDER BY month_no;
    -- Итог возвращается в переменную текущего соединения.
    SET @lr6_last_year_total = v_total, @lr6_status = 'OK';
    DROP TEMPORARY TABLE lr6_month_result;
END$$
DELIMITER ;
```

5 Список процедур на сервере

Запрос из листинга 3 выводит все процедуры базы gibdd, не ограничивая список префиксом lr6. На проверочном сервере обнаружены две разработанные процедуры (рисунок 4). Для просмотра исходного определения дополнительно выполняется SHOW CREATE PROCEDURE.

Листинг 3 — Проверка списка и определений

```
SELECT ROUTINE_NAME, ROUTINE_TYPE, SQL_DATA_ACCESS
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'PROCEDURE'
ORDER BY ROUTINE_NAME;
SHOW CREATE PROCEDURE gibdd.lr6_car_card;
SHOW CREATE PROCEDURE gibdd.lr6_accidents_by_month;
```

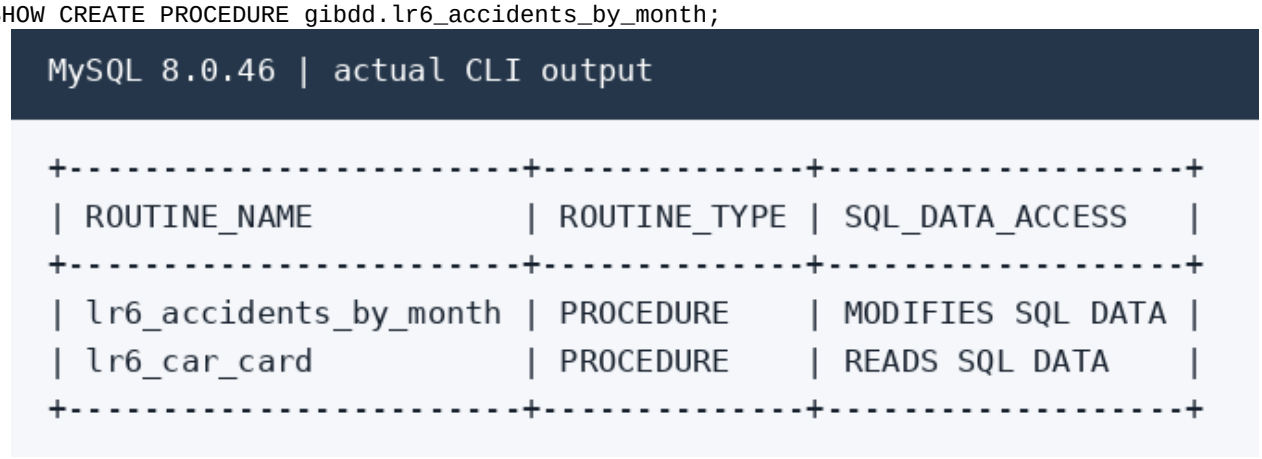


Рисунок 4 — Визуализация фактического вывода списка процедур (CLI)

6 Тестирование и результаты

Тесты выполнены на локальном сервере MySQL 8.0.46 с контрольными данными gibdd. Таблицы 3–5 и рисунки 4–5 основаны на фактическом

выводе клиента mysql; изображения протоколов являются визуализацией текстового вывода, а не снимками интерфейса Workbench. Для окончательного оформления по пунктам 5 и 7 методички необходимо приложить собственные скриншоты выполнения тех же запросов в Workbench. SQL и последовательность съёмки приложены к работе.

Положительные вызовы приведены в листинге 4. CALL и SELECT @-переменных выполняются в одном соединении. Карточка № 1 должна показать два участия, карточка № 9 — ноль; отсутствие участий не считается ошибкой. Для сводки 2025 года проверяется как ветка HAS_DATA, так и NO_DATA, для 2024 года — полностью пустой по ДТП год.

Листинг 4 — Положительные и граничные вызовы

```
CALL gibdd.lr6_car_card(1);
SELECT @lr6_status, @lr6_last_accident_count;
CALL gibdd.lr6_car_card(9);
SELECT @lr6_status, @lr6_last_accident_count;
CALL gibdd.lr6_accidents_by_month(2025);
SELECT @lr6_status, @lr6_last_year_total;
CALL gibdd.lr6_accidents_by_month(2024);
SELECT @lr6_status, @lr6_last_year_total;
CALL gibdd.lr6_accidents_by_month(1901);
CALL gibdd.lr6_accidents_by_month(2155);
```

Таблица 3 — Фактические результаты карточек

carId	Марка; текущий владелец	ДТП	Статус
1	Lada Vesta; Петрова Мария	2	HAS_ACCIDENTS
9	Ford Focus; Новикова Ольга	0	NO_ACCIDENTS

Обе карточки вернули ровно одну строку, @lr6_status = OK, сессионный счётчик равен столбцу accident_count. Для автомобиля № 1 также получены releaseYear = 2020, price = 950000.00, citizenId = 2; для № 9 — 2016, 900000.00, citizenId = 8.

Таблица 4 — Фактическая сводка за 2025 год

Месяц	Число ДТП	Отметка
1	0	NO_DATA
2	0	NO_DATA
3	0	NO_DATA
4	0	NO_DATA
5	0	NO_DATA
6	0	NO_DATA
7	0	NO_DATA
8	1	HAS_DATA
9	2	HAS_DATA
10	2	HAS_DATA
11	0	NO_DATA
12	0	NO_DATA

Сводка содержит 12 строк, сумма accident_count равна 5, @lr6_last_year_total = 5 и @lr6_status = OK. Для 2024, 1901 и 2155 годов получено по 12 строк с нулевыми значениями, NO_DATA и итогом 0. Граничные годы принимаются. Данные совпадают с распределением, указанным в отчёте № 5.

Отрицательные тесты перечислены в таблице 5 и листинге 5. Каждый ошибочный CALL следует запускать отдельно: Workbench может остановить пакет на первой ошибке. Ожидаемая диагностика — ERROR 1644 (45000). Английские сообщения в SQL обеспечивают однозначную проверку независимо от языка интерфейса.

Таблица 5 — Фактические отрицательные тесты

Вызов	Ожидаемая и полученная диагностика
lr6_car_card(NULL)	carId must be a positive integer
lr6_car_card(0)	carId must be a positive integer
lr6_car_card(-1)	carId must be a positive integer
lr6_car_card(999999)	Car with this carId does not exist
lr6_accidents_by_month(NULL)	Year must be between 1901 and 2155
lr6_accidents_by_month(1900)	Year must be between 1901 and 2155
lr6_accidents_by_month(2156)	Year must be between 1901 and 2155

Листинг 5 — Вызовы для ошибочных параметров

```
USE gibdd;
-- Каждый CALL выполнять отдельно: ошибка 1644 / SQLSTATE 45000 ожидаема.
CALL lr6_car_card(NULL);
CALL lr6_car_card(0);
CALL lr6_car_card(-1);
CALL lr6_car_card(999999);
CALL lr6_accidents_by_month(NULL);
CALL lr6_accidents_by_month(1900);
CALL lr6_accidents_by_month(2156);
-- После любой ошибки отдельно проверить сброс сессионного результата:
SELECT @lr6_status, @lr6_last_accident_count, @lr6_last_year_total;
```

```
MySQL 8.0.46 | negative tests

ERROR 1644 (45000) at line 3: carId must be a positive integer
ERROR 1644 (45000) at line 4: carId must be a positive integer
ERROR 1644 (45000) at line 5: carId must be a positive integer
ERROR 1644 (45000) at line 6: Car with this carId does not exist
ERROR 1644 (45000) at line 7: Year must be between 1901 and 2155
ERROR 1644 (45000) at line 8: Year must be between 1901 and 2155
ERROR 1644 (45000) at line 9: Year must be between 1901 and 2155
+-------------+--------------------------+----------------------+
| @lr6_status | @lr6_last_accident_count | @lr6_last_year_total |
+-------------+--------------------------+----------------------+
| ERROR       | NULL                     | NULL                 |
+-------------+--------------------------+----------------------+
```

Рисунок 5 — Визуализация реального протокола ошибочных вызовов (CLI)

В тестовом соединении после ошибок @lr6_status = ERROR, оба итоговых значения NULL. Обработчик SQL-ошибки второй процедуры сработал и повторно передал SIGNAL клиенту. Отдельное искусственное создание ошибки ресурсов сервера не выполнялось; оно не относится к проверке пользовательских параметров.

Автоматическая проверка сравнила результат с ожидаемым: 6 положительных и граничных вызовов и 7 отрицательных вызовов прошли успешно. Дополнительно до и после серии вызовов сравнены все строки пяти базовых таблиц: изменений нет. Полные протоколы и файл результатов включены в комплект. Изменение данных, COMMIT и ROLLBACK для этих процедур не требуются.

7 Вывод

Хранимые процедуры позволяют выразить операции предметной области «ГИБДД» как повторяемые действия сервера. Вместо самостоятельного соединения таблиц инспектор вызывает карточку по идентификатору автомобиля и получает сведения о текущем владельце и зарегистрированных участиях в ДТП. Это снижает вероятность пропустить связи между автомобилем, гражданином и происшествием. При этом

границы смысла результата сохраняются: число участий не доказывает виновность и не восстанавливает историю владения.

Помесячная сводка полезна аналитику тем, что возвращает одинаковую структуру для любого допустимого года. Нулевые месяцы присутствуют в результате, поэтому отсутствие происшествий не смешивается с отсутствием строки отчёта. Цикл реализует последовательный обход календаря; для пяти ДТП 2025 года получены один случай в августе и по два в сентябре и октябре. Эти значения подтверждают согласованность процедуры с запросами предыдущей работы.

Проверка параметров является частью прикладного поведения. Неизвестный автомобиль и некорректный год приводят к явной ошибке, тогда как существующий автомобиль без ДТП и год без происшествий возвращают корректный нулевой результат. Локальные переменные описывают внутреннее состояние алгоритма, а пользовательские сессионные переменные позволяют получить итог после CALL. Их область действия требует использовать одно соединение и проверять статус перед чтением результата.

Положительные, граничные и отрицательные испытания подтвердили работу ветвлений и цикла. Прикладные таблицы остались неизменными, временный результат очищается по завершении работы. Для небольшой учебной базы цикл из 12 запросов нагляден; на большом объёме данных стоит сравнить его с одной групповой выборкой и календарной таблицей и изучить необходимость индекса по accidentDate. Это показывает различие между демонстрацией программных конструкций и выбором решения для производственной нагрузки.

Список использованных источников

1. Аграновский А. В., Боженко В. В., Турнецкая Е. Л. Разработка и администрирование базы данных с открытым исходным кодом: учебно-

методическое пособие. Санкт-Петербург: ГУАП, 2022. Раздел 7; подраздел 7.5, с. 131–133; приложение В, вариант 13.

2. MySQL 8.0 Reference Manual. CREATE PROCEDURE and CREATE FUNCTION Statements; Local Variables; User-Defined Variables; IF Statement; WHILE Statement; SIGNAL Statement; DECLARE ... HANDLER Statement. URL: https://dev.mysql.com/doc/refman/8.0/en/.

3. Курбатов М. П. Отчёт о лабораторной работе № 5 «Создание запросов на сервере MySQL». Вариант 13 — «ГИБДД». ГУАП, 2026. Предоставленный исходный отчёт.

4. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. ГОСТ Р 2.105–2019. Единая система конструкторской документации. Общие требования к текстовым документам.